

ContextDyG: Pair-Conditioned Contextual Frequency Modeling for Continuous-Time Dynamic Link Prediction

Yiming Ai^{1,2}, Peipei Liu³, Yimo Ren^{1,2*}, and Hong Li^{1,2}

¹ Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China
{aiyiming,renyimo,lihong}@iie.ac.cn

² School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China

³ Zhongguancun Laboratory, Beijing, China
peipliu@yeah.net

Abstract. Dynamic link prediction on continuous-time dynamic graphs is important for modeling evolving interactions in social, recommendation, communication, and traffic systems. Recent methods have improved temporal representation learning by exploiting neighbor co-occurrence and interaction frequency, but they still tend to treat historical neighbors as homogeneous evidence. In practice, the same neighbor may act as a local contact of the source node, a shared bridge between two endpoints, a direct historical partner, or a short-term burst around the candidate edge, leading to different implications for future links even with similar occurrence counts. To address this issue, we propose ContextDyG, a pair-conditioned dynamic graph learning framework. ContextDyG introduces Role-aware Contextual Interaction Frequency Encoding (RCIF) to characterize neighbor roles, temporal recency, burstiness, trend, and multi-scale spectral patterns, and Pair-Conditioned Feature Attention Fusion (PC-FAF) to adaptively combine node, edge, time, and contextual-frequency representations for each candidate pair. Experiments on five real-world continuous-time dynamic graph datasets under transductive and inductive settings show that ContextDyG achieves the best average rank across 30 evaluation settings, with average ranks of 1.53 in AP and 1.73 in ROC-AUC. Fine-grained ablations, efficiency analysis, and group-wise case studies further verify the contribution and practicality of the proposed components.

Keywords: Continuous-Time Dynamic Graphs · Dynamic Link Prediction · Contextual Interaction Modeling

1 Introduction

Dynamic graphs describe systems whose entities and relationships evolve over time. They naturally appear in social networks, recommender systems, communication networks, and traffic systems [3, 12, 21, 25]. Compared with discrete-time

* Corresponding author.

dynamic graphs, continuous-time dynamic graphs (CTDGs) preserve each interaction as a timestamped event and avoid the information loss caused by coarse snapshots [9, 13, 23, 24]. This paper focuses on dynamic link prediction on CTDGs, which predicts whether two nodes will interact at a future time based only on previous interactions [10].

Recent CTDG methods have made steady progress by modeling temporal evolution with memory networks [5, 11, 14], temporal random walks [4, 18], temporal point processes [1, 2], differential equations [7, 8, 13], recurrent neural networks [17, 22], contrastive learning [16], and self-attention mechanisms [15, 24]. Beyond these temporal architectures, DyGFormer and FreeDyG show that neighbor co-occurrence and interaction frequency are strong signals for link prediction [13, 24]. Their success suggests that a candidate edge should be understood through the historical neighborhoods around its two endpoints, rather than through isolated node identities alone.

However, frequency-based neighborhood modeling is still not sufficiently contextual. Consider a candidate edge between Alice and Bob in a social network. A mutual friend Carol may be Alice’s long-term local contact, a bridge recently introduced through Bob, or a direct historical mediator between the two users. These cases may have similar co-occurrence counts but different implications for the future link. A similar issue appears in recommendation or communication networks: the same shared item, user, or device can indicate stable preference, cross-side influence, or a short-term burst. Existing frequency encodings mostly answer “how often a neighbor appears”, but do not explicitly answer “what role this neighbor plays for the current node pair” or “whether its influence is stable or bursty”. Moreover, node, edge, time, and structural features are often fused by concatenation or static attention, although their usefulness varies across node pairs and temporal contexts.

To address these limitations, we propose ContextDyG, a CTDG link prediction framework with two pair-conditioned components. Role-aware Contextual Interaction Frequency Encoding (RCIF) enriches interaction frequency with local and cross-side counts, direct historical matches, shared-neighbor roles, source-target asymmetry, temporal recency, burstiness, trends, and temporal overlap. A lightweight multi-scale spectral filter further separates stable low-frequency contextual patterns from bursty high-frequency patterns. Pair-Conditioned Feature Attention Fusion (PC-FAF) then treats node, edge, time, and RCIF representations as modality tokens and uses the current source-destination context to adaptively fuse them. In this way, ContextDyG models not only which neighbors appear, but also how they participate in the formation of a candidate edge.

We evaluate ContextDyG on five public real-world datasets, WIKI, MOOC, Enron, UCI, and CanParl, under transductive and inductive settings with random, historical, and inductive negative sampling. The results show that ContextDyG achieves competitive performance against strong CTDG baselines, with clearer advantages in settings where negative edges are historically plausible. We also provide fine-grained ablations, efficiency analysis, and group-wise case stud-

ies to examine which components contribute to the gains and when contextual modeling is most useful.

The main contributions of this paper are summarized as follows:

1. We propose ContextDyG, a continuous-time dynamic graph learning framework that models historical neighborhoods from a pair-conditioned contextual perspective.
2. We introduce RCIF, a role-aware contextual interaction frequency encoding module that captures neighbor roles, temporal dynamics, and stable or bursty frequency patterns.
3. We design PC-FAF, a pair-conditioned feature fusion module that adaptively integrates node, edge, time, and contextual frequency representations.
4. We conduct comprehensive experiments with multiple negative sampling strategies, fine-grained ablations, efficiency analysis, and group-wise case studies to validate the effectiveness and practicality of ContextDyG.

2 Related Work

2.1 Continuous-Time Dynamic Graph Learning

Dynamic graph learning methods can be broadly divided into discrete-time dynamic graph methods and continuous-time dynamic graph methods. Discrete-time methods represent graph evolution as a sequence of snapshots and then apply static or recurrent graph models across snapshots [9, 23, 25]. This formulation is intuitive, but the discretization process may hide interactions occurring between adjacent snapshots. CTDG methods instead model the graph as a stream of timestamped events and directly learn from irregular interactions [10, 13, 24]. This event-based formulation is more suitable for link prediction when the exact order and time interval of interactions matter.

Existing CTDG methods differ mainly in how they summarize historical interactions. Memory-based methods, such as JODIE, DyRep, and TGN, maintain dynamic node states and update them after each observed interaction [5, 11, 14]. Temporal random-walk based methods sample time-respecting walks to capture high-order temporal structures [4, 18]. Other methods use temporal point processes [1, 2], differential equations [7, 8, 13], recurrent neural networks [17, 22], contrastive learning [16], or attention mechanisms [15, 24]. These methods provide powerful temporal encoders, but the historical neighborhood is often compressed without explicitly asking what role each neighbor plays for the candidate edge.

2.2 Neighbor Co-occurrence and Frequency Modeling

Neighbor co-occurrence and interaction frequency have recently been shown to be useful for dynamic link prediction [10, 13, 24]. DyGFormer introduces neighbor co-occurrence encoding to record whether historical neighbors appear around both endpoints [24]. FreeDyG further introduces node interaction frequency and

frequency-enhanced sequence modeling for CTDGs [13]. These methods move beyond raw timestamps and show that the neighborhood around a candidate pair contains valuable structural evidence.

However, a frequency value alone does not explain how a neighbor participates in the current prediction. A neighbor that appears only in the source history, a neighbor shared by both endpoints, and a neighbor that recently bursts on the target side may have different semantic meanings even when their counts are similar. Treating them as similar frequency events may weaken the model’s ability to distinguish structurally plausible but false links, especially under historical and inductive negative sampling. Our proposed RCIF extends interaction-frequency modeling by incorporating role-aware statistics, temporal dynamics, and multi-scale frequency patterns.

2.3 Feature Fusion in Dynamic Graph Models

Dynamic graph models often combine multiple types of information, including node attributes, edge attributes, time encodings, structural encodings, and interaction-frequency features. A common strategy is to concatenate these features and project them into a unified representation space. Some methods further use attention mechanisms to aggregate temporal neighbors or combine sequence representations [15, 16, 22, 24]. Although effective, these fusion strategies are usually static or pair-agnostic: the relative importance of node, edge, time, and structural features is not explicitly conditioned on the candidate edge being predicted.

In dynamic link prediction, the useful evidence can change from one candidate edge to another. Repeated interactions may dominate one prediction, shared-neighbor roles may dominate another, and short-term temporal bursts may dominate a third. Therefore, a fixed fusion strategy may be insufficient. PC-FAF uses the current source-destination context as a query to adaptively fuse node, edge, time, and RCIF representations, enabling pair-conditioned feature selection for dynamic link prediction.

3 Methodology

3.1 Overview

Given a candidate interaction (u, v, t) , ContextDyG estimates the probability that endpoint nodes u and v will interact at time t using only previous interactions. As shown in Fig. 1, the model follows four layers: input, encoding, feature attention fusion, and output. The following subsections describe these layers in order.

3.2 Input Layer

For each candidate pair (u, v, t) , we sample the most recent L historical interactions before time t . The source-side history is denoted by

$$\mathcal{N}_u^t = \{(x_i, e_i, t_i)\}_{i=1}^L, \quad t_i < t, \quad (1)$$

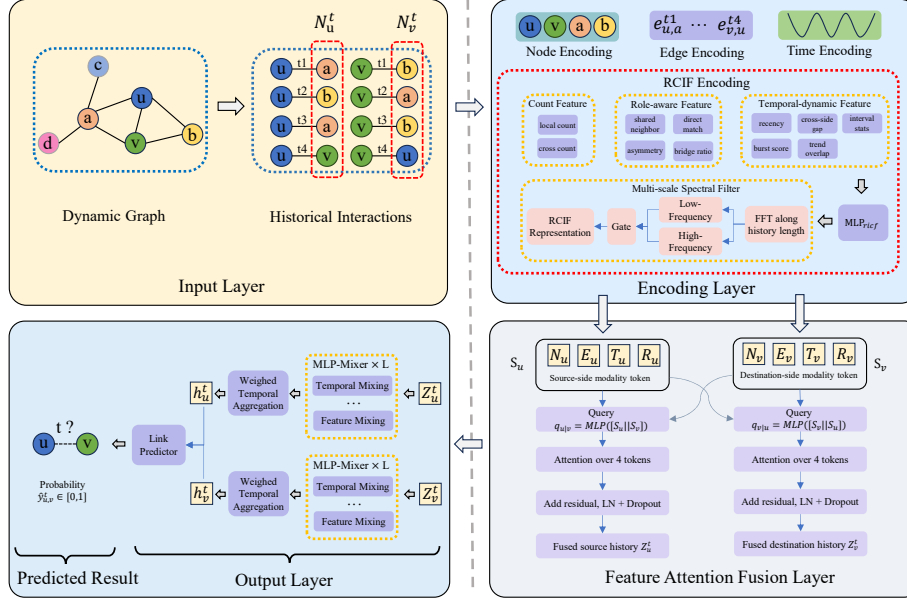


Fig. 1. The overview of ContextDyG. Given a candidate edge, ContextDyG retrieves historical neighborhoods, constructs role-aware contextual interaction features, applies multi-scale spectral filtering, and fuses node, edge, time, and RCIF representations with pair-conditioned attention for link prediction.

where x_i is the i -th historical neighbor of u , e_i is the corresponding edge feature, and t_i is the interaction time. The destination-side history $\mathcal{N}_v^t = \{(y_i, e'_i, t'_i)\}_{i=1}^L$ is defined in the same way. This layer only provides the two endpoint histories; all representations are constructed in the following encoding layer.

3.3 Encoding Layer

The encoding layer maps each historical interaction into four representations: node, edge, time, and RCIF representations. For a side $a \in \{u, v\}$, let $\mathcal{N}_a^t$ be its historical sequence and let i index a historical position. The node representation $\mathbf{n}_{a,i}$ is obtained from the raw feature or embedding of the historical neighbor, the edge representation $\mathbf{e}_{a,i}$ is obtained from the raw edge feature, and the time representation $\mathbf{t}_{a,i}$ is produced from the time interval $\Delta t_{a,i} = t - t_{a,i}$ using a cosine time encoder [15, 22, 24]. The fourth representation, $\mathbf{r}_{a,i}$, is produced by RCIF to describe the contextual role of the historical neighbor.

We next describe RCIF on the source side; the destination side is computed symmetrically. For a source-side historical neighbor x , RCIF first counts how often x appears in the source-side and destination-side histories:

$$C_u(x) = \sum_{(x_i, \cdot, \cdot) \in \mathcal{N}_u^t} \mathbb{I}(x_i = x), \quad C_v(x) = \sum_{(y_i, \cdot, \cdot) \in \mathcal{N}_v^t} \mathbb{I}(y_i = x). \quad (2)$$

Here $C_u(x)$ measures local occurrence around the source node, while $C_v(x)$ measures cross-side occurrence around the destination node.

Based on these counts, RCIF builds three groups of features. The count group contains transformed local and cross-side counts. The role-aware group contains a shared-neighbor indicator $\mathbb{I}(C_u(x) > 0 \wedge C_v(x) > 0)$, a direct-match indicator $\mathbb{I}(x = v)$, a source-target asymmetry score $\log(1 + C_u(x)) - \log(1 + C_v(x))$, and a bridge ratio $C_v(x)/(C_u(x) + C_v(x) + \epsilon)$, where ϵ is a small constant for numerical stability. The temporal-dynamic group contains recency, cross-side time gap, interval statistics, burst score, and trend-overlap features. These features describe whether x is recent, regular, bursty, or temporally aligned across the two endpoint histories. The raw RCIF vector is written as

$$\mathbf{r}_{u,x}^{raw} = [\mathbf{r}_{u,x}^{count} \parallel \mathbf{r}_{u,x}^{role} \parallel \mathbf{r}_{u,x}^{time}], \quad (3)$$

where $\parallel$ denotes concatenation. Therefore, RCIF describes not only how often a neighbor appears, but also what role it plays and how its activity evolves over time.

The raw RCIF vectors along the history are projected by an MLP into a sequence $\mathbf{H}^{rcif} \in \mathbb{R}^{L \times d}$, where d is the hidden dimension. To separate stable and bursty contextual patterns, we apply a lightweight spectral filter along the history length. Let $\mathbf{H}^{low}$ and $\mathbf{H}^{high}$ be the low-frequency and high-frequency components obtained from the Fourier spectrum, and let $\mathbf{g} \in [0, 1]^d$ be a learnable gate. The final RCIF sequence is

$$\tilde{\mathbf{H}}^{rcif} = \mathbf{H}^{rcif} + \text{Dropout}((1 - \mathbf{g})\mathbf{H}^{low} + \mathbf{g}\mathbf{H}^{high}). \quad (4)$$

Low-frequency components capture persistent contextual relations, while high-frequency components capture short-term bursts, following the frequency-domain intuition used in temporal representation learning [6, 13, 19, 20, 26]. In implementation, FFT is applied along the history-length dimension. The first quarter of frequency bands is treated as the low-frequency part, and the remaining bands are treated as high-frequency components. The gate is generated from the mean sequence representation and applied channel-wise.

These RCIF statistics are not dataset-specific attributes. They are computed from temporal neighborhoods of the current candidate pair, so the same encoder can be applied to unseen nodes and different CTDG datasets.

3.4 Feature Attention Fusion Layer

After the encoding layer, each historical position has four representations. For side $a \in \{u, v\}$ and position i , we treat them as four modality tokens:

$$\mathbf{M}_{a,i} = [\mathbf{n}_{a,i}, \mathbf{e}_{a,i}, \mathbf{t}_{a,i}, \mathbf{r}_{a,i}] \in \mathbb{R}^{4 \times d}, \quad (5)$$

where $\mathbf{r}_{a,i}$ is the i -th vector in $\tilde{\mathbf{H}}^{rcif}$. Instead of using a fixed concatenation, PC-FAF generates a pair-conditioned query. We first compute a side summary $\mathbf{s}_a = \text{MeanPool}(\{\mathbf{M}_{a,i}\}_{i=1}^L)$, and then form two directional queries:

$$\mathbf{q}_{u|v} = \text{MLP}([\mathbf{s}_u \parallel \mathbf{s}_v]), \quad \mathbf{q}_{v|u} = \text{MLP}([\mathbf{s}_v \parallel \mathbf{s}_u]). \quad (6)$$

Here $\mathbf{q}_{u|v}$ means that the source-side fusion is conditioned on the destination-side context, and $\mathbf{q}_{v|u}$ is defined analogously.

For source-side fusion, the query is $\mathbf{q}_{u|v}$, while the key and value are produced from the source-side tokens $\mathbf{M}_{u,i}$. The fused source representation is

$$\mathbf{z}_{u,i} = \text{Dropout} \left(\text{LayerNorm} \left(\text{MHA}(\mathbf{q}_{u|v}, \mathbf{M}_{u,i}, \mathbf{M}_{u,i}) + W_c \text{Concat}(\mathbf{M}_{u,i}) \right) \right), \quad (7)$$

where MHA uses learnable query, key, and value projections internally, and W_c maps the concatenated four modality tokens back to dimension d as a residual path. Thus, attention is performed over the four modality tokens at each historical position, rather than over different historical positions. The destination-side representation $\mathbf{z}_{v,i}$ is computed in the same way using $\mathbf{q}_{v|u}$ and $\mathbf{M}_{v,i}$. This layer outputs two fused historical sequences $\mathbf{Z}_u^t$ and $\mathbf{Z}_v^t$.

3.5 Output Layer

The fused historical sequences $\mathbf{Z}_u^t$ and $\mathbf{Z}_v^t$ are fed into MLP-Mixer blocks, which mix information along the temporal and feature dimensions following FreeDyG [13]. A learnable weighted aggregation layer then produces the final temporal node representations $\mathbf{h}_u^t$ and $\mathbf{h}_v^t$. The predicted link probability is

$$\hat{y}_{u,v}^t = \sigma(\text{MLP}([\mathbf{h}_u^t || \mathbf{h}_v^t])). \quad (8)$$

The model is optimized with binary cross-entropy loss. The main additional costs come from RCIF comparison, spectral filtering, and modality attention. We analyze these costs empirically in the efficiency study.

4 Experiments

In this section, we evaluate ContextDyG on dynamic link prediction. We first introduce the datasets, baselines, evaluation protocols, and implementation details. Then, we report the main comparison results, fine-grained ablation studies, and efficiency analysis. Since dynamic link prediction is sensitive to negative sampling strategies, we evaluate all models under random, historical, and inductive negative sampling.

4.1 Datasets

We conduct experiments on five real-world continuous-time dynamic graph datasets, including WIKI, MOOC, Enron, UCI, and CanParl, following the benchmark collection and evaluation protocol of previous dynamic link prediction studies [10, 13, 24]. These datasets cover different domains, such as online editing, online education, email communication, social interaction, and parliamentary voting. The statistics of the datasets are shown in Table 1.

To avoid temporal leakage, all datasets are split chronologically into training, validation, and test sets. In the inductive setting, edges involving held-out new nodes are removed from training, so the model is evaluated on interactions containing nodes unseen during training.

Table 1. Statistics of the datasets.

Dataset	Nodes	Edges	Duration
WIKI	9,227	157,474	1 month
MOOC	7,144	411,749	17 months
Enron	184	125,235	3 years
UCI	1,899	59,835	196 days
CanParl	734	74,478	14 years

4.2 Baselines

We compare ContextDyG with representative continuous-time dynamic graph models, including TGAT [22], TCL [16], DyGFormer [24], and FreeDyG [13]. TGAT uses temporal attention to aggregate historical neighbors. TCL models temporal dependencies through contrastive learning. DyGFormer uses patch-based sequence modeling and neighbor co-occurrence encoding. FreeDyG introduces frequency-enhanced interaction modeling and is one of the strongest recent baselines. For results reproduced in our environment, we use the same data splits and negative sampling protocol. For reported baseline results, we follow the corresponding benchmark settings whenever available [10].

4.3 Evaluation Protocol

We split each dataset chronologically into training, validation, and test sets with ratios of 70%, 15%, and 15%, as widely adopted in CTDG evaluation [10, 13, 24]. All models only use interactions before the prediction time. We evaluate both transductive and inductive settings. In the transductive setting, test edges may involve nodes observed during training. In the inductive setting, we first collect nodes that appear after the training cutoff and randomly sample 10% of all nodes from them as held-out new nodes with a fixed seed. All training edges involving these nodes are removed. Validation and test edges containing at least one held-out node form the inductive validation and test sets.

We consider three negative sampling strategies [10]. For each positive edge, one negative edge is sampled. Random negative sampling keeps the source node and randomly samples a destination node. Historical negative sampling samples node pairs that appeared before the current batch time but do not appear in the current batch, making negatives historically plausible but currently inactive. Inductive negative sampling further removes node pairs observed before the cutoff time: the training cutoff is used for validation, and the validation cutoff is used for testing. Thus, inductive negatives are plausible edges not observed in the previous observation window. In implementation, historical and inductive negative sampling explicitly exclude positive edges in the current batch; random negative sampling follows the standard random destination replacement protocol.

Table 2. AP for dynamic link prediction under transductive and inductive settings.

NSS	Dataset	TGAT	TCL	DyGFormer	FreeDyG	ContextDyG
(a) Transductive Setting						
rnd	WIKI	96.95 $\pm$ 0.17	96.47 $\pm$ 0.16	98.82 $\pm$ 0.02	99.26 $\pm$ 0.01	99.33 $\pm$ 0.01
	MOOC	85.44 $\pm$ 0.76	81.12 $\pm$ 0.43	87.23 $\pm$ 0.45	89.61 $\pm$ 0.19	89.53 $\pm$ 0.05
	Enron	72.58 $\pm$ 0.79	79.70 $\pm$ 0.71	92.47 $\pm$ 0.12	92.51 $\pm$ 0.05	92.85 $\pm$ 0.03
	UCI	79.55 $\pm$ 0.83	88.12 $\pm$ 2.73	95.76 $\pm$ 0.15	96.28 $\pm$ 0.11	95.99 $\pm$ 0.01
	CanParl	57.54 $\pm$ 2.84	59.11 $\pm$ 0.58	70.50 $\pm$ 0.74	74.83 $\pm$ 2.86	78.22 $\pm$ 0.35
hist	WIKI	87.01 $\pm$ 0.19	88.75 $\pm$ 0.27	82.23 $\pm$ 2.54	91.59 $\pm$ 0.57	85.02 $\pm$ 3.77
	MOOC	82.19 $\pm$ 0.62	77.06 $\pm$ 0.41	85.85 $\pm$ 0.66	86.71 $\pm$ 0.81	86.14 $\pm$ 0.25
	Enron	64.07 $\pm$ 1.05	70.66 $\pm$ 0.39	75.63 $\pm$ 0.73	<u>78.87 $\pm$ 0.82</u>	82.15 $\pm$ 0.54
	UCI	68.27 $\pm$ 1.37	80.25 $\pm$ 2.74	82.17 $\pm$ 0.82	<u>86.10 $\pm$ 1.19</u>	87.22 $\pm$ 0.21
	CanParl	55.79 $\pm$ 0.67	57.95 $\pm$ 0.31	<u>67.26 $\pm$ 7.03</u>	61.09 $\pm$ 16.03	74.89 $\pm$ 7.13
ind	WIKI	87.00 $\pm$ 0.16	86.76 $\pm$ 0.72	78.29 $\pm$ 5.38	90.05 $\pm$ 0.79	85.42 $\pm$ 5.85
	MOOC	75.95 $\pm$ 0.64	74.65 $\pm$ 0.54	81.24 $\pm$ 0.69	83.01 $\pm$ 0.87	85.18 $\pm$ 0.13
	Enron	63.94 $\pm$ 1.36	71.29 $\pm$ 0.32	77.41 $\pm$ 0.89	<u>77.81 $\pm$ 0.65</u>	78.80 $\pm$ 0.88
	UCI	68.67 $\pm$ 0.84	76.01 $\pm$ 1.11	72.25 $\pm$ 1.71	<u>82.35 $\pm$ 0.73</u>	82.68 $\pm$ 0.57
	CanParl	59.08 $\pm$ 4.82	62.28 $\pm$ 0.88	71.84 $\pm$ 2.51	56.95 $\pm$ 13.68	<u>69.71 $\pm$ 5.46</u>
(b) Inductive Setting						
rnd	WIKI	96.22 $\pm$ 0.07	96.22 $\pm$ 0.17	98.59 $\pm$ 0.03	98.97 $\pm$ 0.01	98.85 $\pm$ 0.01
	MOOC	85.50 $\pm$ 0.19	80.60 $\pm$ 0.22	86.96 $\pm$ 0.43	<u>87.75 $\pm$ 0.62</u>	89.04 $\pm$ 0.09
	Enron	67.05 $\pm$ 1.51	76.14 $\pm$ 0.79	89.76 $\pm$ 0.34	<u>89.69 $\pm$ 0.17</u>	89.28 $\pm$ 0.25
	UCI	79.54 $\pm$ 0.48	87.36 $\pm$ 2.03	94.54 $\pm$ 0.12	<u>94.85 $\pm$ 0.10</u>	95.06 $\pm$ 0.04
	CanParl	52.08 $\pm$ 0.58	<u>56.53 $\pm$ 0.49</u>	52.73 $\pm$ 0.02	53.31 $\pm$ 1.80	57.78 $\pm$ 0.42
hist	WIKI	84.17 $\pm$ 0.22	82.20 $\pm$ 2.18	71.42 $\pm$ 4.43	82.78 $\pm$ 0.30	86.92 $\pm$ 6.50
	MOOC	76.73 $\pm$ 0.29	74.27 $\pm$ 0.53	80.82 $\pm$ 0.30	81.63 $\pm$ 0.33	84.63 $\pm$ 0.20
	Enron	61.40 $\pm$ 1.31	67.11 $\pm$ 0.62	67.07 $\pm$ 0.62	<u>73.01 $\pm$ 0.88</u>	78.48 $\pm$ 0.63
	UCI	70.52 $\pm$ 0.93	76.71 $\pm$ 1.00	72.13 $\pm$ 1.87	<u>82.35 $\pm$ 0.39</u>	83.56 $\pm$ 0.46
	CanParl	53.30 $\pm$ 0.93	57.76 $\pm$ 1.20	53.13 $\pm$ 0.65	<u>52.34 $\pm$ 0.87</u>	56.69 $\pm$ 0.22
ind	WIKI	84.17 $\pm$ 0.22	82.20 $\pm$ 2.18	71.42 $\pm$ 4.43	87.54 $\pm$ 0.26	86.92 $\pm$ 6.50
	MOOC	76.72 $\pm$ 0.30	74.28 $\pm$ 0.53	80.82 $\pm$ 0.30	81.41 $\pm$ 0.31	84.63 $\pm$ 0.20
	Enron	61.40 $\pm$ 1.30	67.11 $\pm$ 0.62	67.07 $\pm$ 0.62	<u>72.85 $\pm$ 0.81</u>	78.48 $\pm$ 0.63
	UCI	70.49 $\pm$ 0.93	76.65 $\pm$ 0.99	72.13 $\pm$ 1.86	<u>82.06 $\pm$ 0.58</u>	83.56 $\pm$ 0.47
	CanParl	53.23 $\pm$ 0.97	57.66 $\pm$ 1.00	52.89 $\pm$ 0.54	52.28 $\pm$ 0.76	<u>56.39 $\pm$ 0.18</u>
Avg. Rank		4.23	3.70	3.37	<u>2.17</u>	1.53

For reproduced experiments, all models use the same data splits and fixed evaluation negative samples. We report Average Precision (AP) and ROC-AUC as the main metrics.

4.4 Main Results

Table 2 reports the AP results under different negative sampling strategies. Table 3 reports the corresponding ROC-AUC results. Overall, ContextDyG does not win every individual setting, but it achieves the best overall average rank across all transductive/inductive and negative sampling protocols. The improvement is more obvious under historical and inductive negative sampling, where negative samples are structurally and temporally harder than random negatives. This suggests that modeling pair-specific contextual roles and temporal burstiness is useful for distinguishing plausible but false interactions.

Table 3. ROC-AUC for dynamic link prediction under transductive and inductive settings.

NSS	Dataset	TGAT	TCL	DyGFormer	FreeDyG	ContextDyG
(a) Transductive Setting						
rnd	WIKI	96.67 $\pm$ 0.07	95.84 $\pm$ 0.18	98.91 $\pm$ 0.02	99.41 $\pm$ 0.01	99.29 $\pm$ 0.01
	MOOC	87.11 $\pm$ 0.19	83.12 $\pm$ 0.18	87.91 $\pm$ 0.58	89.93 $\pm$ 0.35	89.84 $\pm$ 0.08
	Enron	68.89 $\pm$ 1.10	75.74 $\pm$ 0.72	93.33 $\pm$ 0.13	94.01 $\pm$ 0.11	94.83 $\pm$ 0.02
	UCI	78.53 $\pm$ 0.74	87.82 $\pm$ 1.36	94.49 $\pm$ 0.26	95.00 $\pm$ 0.21	95.28 $\pm$ 0.06
	CanParl	61.76 $\pm$ 4.76	62.52 $\pm$ 0.44	78.35 $\pm$ 1.56	83.39 $\pm$ 1.29	85.60 $\pm$ 0.55
hist	WIKI	82.87 $\pm$ 0.22	85.76 $\pm$ 0.46	78.80 $\pm$ 1.95	82.78 $\pm$ 0.30	83.38 $\pm$ 1.42
	MOOC	80.81 $\pm$ 0.67	72.09 $\pm$ 0.56	87.04 $\pm$ 0.35	88.32 $\pm$ 0.99	83.77 $\pm$ 0.42
	Enron	61.85 $\pm$ 1.43	67.95 $\pm$ 0.88	76.55 $\pm$ 0.52	75.74 $\pm$ 0.72	80.93 $\pm$ 0.16
	UCI	58.89 $\pm$ 1.57	72.25 $\pm$ 3.46	76.97 $\pm$ 0.24	80.38 $\pm$ 0.26	80.54 $\pm$ 0.33
	CanParl	60.72 $\pm$ 2.40	59.30 $\pm$ 2.04	75.34 $\pm$ 6.43	69.52 $\pm$ 16.34	81.04 $\pm$ 4.06
ind	WIKI	81.93 $\pm$ 0.22	82.19 $\pm$ 0.48	75.09 $\pm$ 3.70	82.74 $\pm$ 0.32	79.22 $\pm$ 2.34
	MOOC	73.18 $\pm$ 0.33	70.36 $\pm$ 0.37	80.76 $\pm$ 0.76	78.47 $\pm$ 0.94	76.99 $\pm$ 0.24
	Enron	60.45 $\pm$ 2.12	67.64 $\pm$ 0.86	74.07 $\pm$ 0.64	77.27 $\pm$ 0.61	78.69 $\pm$ 0.16
	UCI	60.80 $\pm$ 1.01	70.05 $\pm$ 1.86	65.96 $\pm$ 1.18	75.39 $\pm$ 0.57	74.12 $\pm$ 0.76
	CanParl	64.06 $\pm$ 4.95	66.11 $\pm$ 2.07	77.92 $\pm$ 2.88	62.07 $\pm$ 13.88	75.36 $\pm$ 3.44
(b) Inductive Setting						
rnd	WIKI	95.90 $\pm$ 0.09	95.57 $\pm$ 0.20	98.48 $\pm$ 0.03	99.01 $\pm$ 0.02	98.84 $\pm$ 0.01
	MOOC	86.84 $\pm$ 0.17	81.43 $\pm$ 0.19	87.62 $\pm$ 0.51	87.01 $\pm$ 0.74	89.57 $\pm$ 0.08
	Enron	64.63 $\pm$ 1.74	72.33 $\pm$ 0.99	90.69 $\pm$ 0.26	89.51 $\pm$ 0.20	89.50 $\pm$ 0.30
	UCI	77.64 $\pm$ 0.38	84.49 $\pm$ 1.82	92.63 $\pm$ 0.13	93.01 $\pm$ 0.08	93.79 $\pm$ 0.04
	CanParl	52.91 $\pm$ 1.19	57.53 $\pm$ 0.28	52.49 $\pm$ 0.77	55.66 $\pm$ 1.29	60.76 $\pm$ 1.90
hist	WIKI	78.38 $\pm$ 0.20	79.79 $\pm$ 0.96	70.33 $\pm$ 0.25	82.08 $\pm$ 0.32	83.89 $\pm$ 3.04
	MOOC	74.08 $\pm$ 0.27	69.82 $\pm$ 0.32	80.77 $\pm$ 0.63	81.52 $\pm$ 0.37	76.45 $\pm$ 0.04
	Enron	57.84 $\pm$ 2.18	64.06 $\pm$ 1.02	65.78 $\pm$ 0.42	70.09 $\pm$ 0.65	76.14 $\pm$ 0.01
	UCI	62.32 $\pm$ 1.18	70.46 $\pm$ 1.94	65.55 $\pm$ 1.01	76.01 $\pm$ 0.75	74.60 $\pm$ 0.57
	CanParl	54.37 $\pm$ 1.47	59.15 $\pm$ 1.32	52.38 $\pm$ 0.49	53.85 $\pm$ 0.35	59.67 $\pm$ 1.63
ind	WIKI	78.38 $\pm$ 0.20	79.79 $\pm$ 0.96	68.33 $\pm$ 2.82	83.17 $\pm$ 0.31	73.89 $\pm$ 3.04
	MOOC	74.07 $\pm$ 0.27	69.83 $\pm$ 0.32	80.77 $\pm$ 0.63	75.81 $\pm$ 0.69	76.45 $\pm$ 0.04
	Enron	57.83 $\pm$ 2.18	64.05 $\pm$ 1.02	65.79 $\pm$ 0.42	68.79 $\pm$ 0.91	75.00 $\pm$ 0.02
	UCI	62.29 $\pm$ 1.17	70.42 $\pm$ 1.93	65.58 $\pm$ 1.00	73.41 $\pm$ 0.88	73.59 $\pm$ 0.58
	CanParl	54.38 $\pm$ 1.39	59.13 $\pm$ 1.12	52.34 $\pm$ 0.40	53.62 $\pm$ 0.62	59.49 $\pm$ 1.60
Avg. Rank		4.27	3.70	3.13	<u>2.17</u>	1.73

It is worth noting that ContextDyG does not necessarily dominate every dataset under every protocol. In relatively easy random negative sampling settings, simple frequency-based or patch-based models may already be sufficient. However, ContextDyG shows stronger robustness under harder negative sampling settings, which better reflects its ability to model contextual interaction patterns.

The results on CanParl further illustrate the motivation of ContextDyG. Parliamentary interactions are not only repeated contacts between two members, but are also affected by shared political contexts and bridge-like neighbors. FreeDyG models interaction frequency effectively, but its frequency signal is less explicit about whether a historical neighbor is local, shared, or asymmetric with respect to the current pair. ContextDyG improves the average rank by adding such role-aware contextual information and by adapting the feature fusion to each candidate edge.

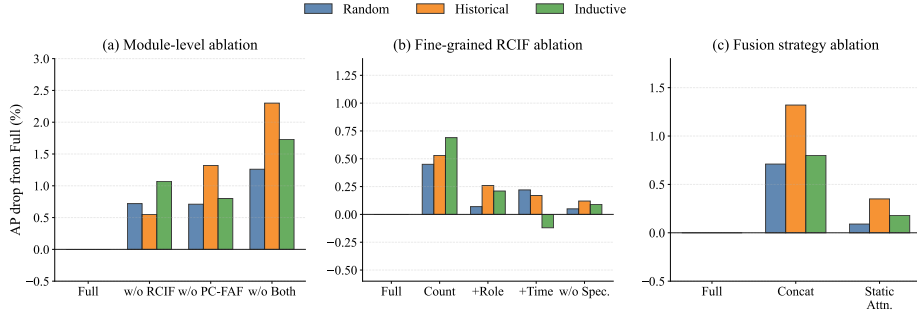


Fig. 2. Ablation study on UCI under random, historical, and inductive negative sampling. The full model is placed first in each panel. Larger drops indicate more important components.

4.5 Ablation Study

To better understand the contribution of each design choice, we conduct fine-grained ablation studies on UCI under random, historical, and inductive negative sampling. Instead of only removing the two main modules, we further decompose RCIF into count-based, role-aware, temporal, and spectral components, and compare PC-FAF with simpler fusion strategies. We use UCI for this controlled study because it is a moderate-scale benchmark where all variants can be evaluated under three negative sampling protocols with manageable cost.

Fig. 2 summarizes three groups of ablations. The module-level results show that removing RCIF or replacing PC-FAF with concatenation consistently hurts performance, and removing both components leads to the largest drop. The RCIF variants show that simple counts already provide useful frequency evidence, while role-aware and temporal features further improve the representation. The spectral filter brings smaller but stable gains on UCI, suggesting that this dataset is driven more by contextual roles than by strong burstiness. The fusion comparison shows that pair-conditioned fusion is more effective than static attention or fixed concatenation, confirming that the useful feature channel changes across candidate node pairs.

4.6 Efficiency Analysis

ContextDyG introduces additional computations for RCIF construction, multi-scale spectral filtering, and pair-conditioned feature fusion. Therefore, we further compare its efficiency with strong baselines. The goal of this experiment is not to claim universal runtime superiority, but to show that the added contextual modules do not necessarily introduce prohibitive overhead in practice. We report the number of trainable parameters, training time per epoch, AP, and ROC-AUC.

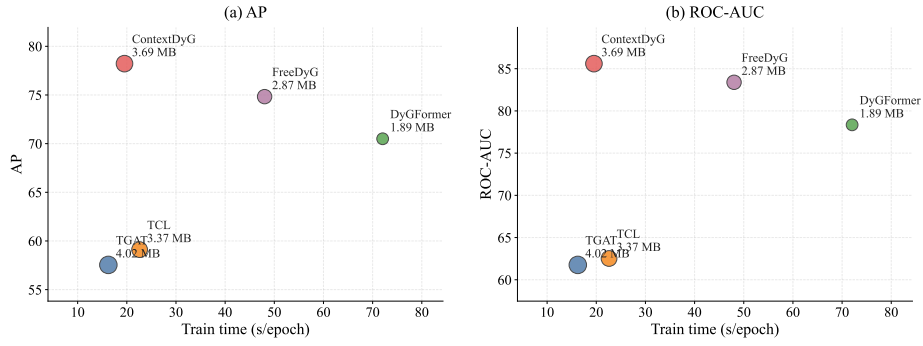


Fig. 3. Efficiency-performance trade-off on CanParl under transductive random negative sampling. The x-axis is training time per epoch, the y-axis is AP or ROC-AUC, and the bubble radius is proportional to the number of parameters in MB.

Although ContextDyG has more trainable parameters than FreeDyG, the wall-clock training time is not determined only by parameter count. In our implementation, ContextDyG uses a fixed short history length of $L = 32$, computes RCIF statistics with batched tensor operations, and applies feature attention over only four modality tokens at each historical position. Thus, the added contextual modules are mostly executed as dense GPU-friendly operations with small token sizes. By contrast, the measured runtime of a baseline also depends on its default sequence length, sampling cost, feature construction pipeline, and implementation details. Therefore, the efficiency comparison should be interpreted as an empirical runtime comparison under the same hardware and evaluation protocol, rather than as a claim that RCIF is theoretically cheaper than the original frequency encoder.

Theoretically, RCIF requires $O(BL^2)$ operations to compare historical neighbors, where B is the batch size and L is the sampled history length. The spectral filter costs $O(BLd \log L)$, and PC-FAF costs $O(BLm^2d)$, where d is the hidden dimension and $m = 4$ is the number of feature modalities. Since L is fixed to a small value in practice, the additional overhead is bounded. The efficiency results in Fig. 3 show that ContextDyG achieves a favorable empirical trade-off on CanParl: it improves predictive performance while keeping the measured per-epoch runtime lower than FreeDyG in our implementation. We do not claim that this speed advantage will hold under all hyperparameter choices; for example, increasing L would enlarge the RCIF comparison cost.

4.7 Case Study: Group-wise Error Analysis

We conduct a group-wise analysis on CanParl under historical negative sampling to further understand the behavior of ContextDyG. For each test sample, we compute two historical statistics before the prediction time: the number of shared historical neighbors between the source and target nodes, and the historical

Table 4. Group-wise AP on CanParl under historical negative sampling.

Group	Criterion	#Samples	DyGFormer	FreeDyG	ContextDyG
Low context	shared neighbors ≤ 35	6,684	93.01	92.78	93.97
Medium context	shared neighbors = 36–89	6,754	63.55	69.32	77.96
High context	shared neighbors > 89	6,788	53.62	58.45	68.95
Low frequency	pair frequency ≤ 1	12,689	77.26	71.77	80.97
Medium frequency	pair frequency = 2	4,523	64.66	79.68	85.79
High frequency	pair frequency > 2	3,014	76.15	85.87	92.34

interaction frequency of the node pair. The former measures the strength of neighborhood context, while the latter reflects the degree of pairwise repetition.

To avoid manually selected thresholds, we divide test samples according to the empirical distribution of these statistics. For shared historical neighbors, the 33rd and 66th percentiles are 35 and 89, respectively; therefore, we define low-, medium-, and high-context groups as ≤ 35 , $36 \sim 89$, and > 89 . For pair frequency, the distribution is discrete and concentrated on small values, so we use ≤ 1 , $= 2$, and > 2 to form low-, medium-, and high-frequency groups.

Table 4 shows the group-wise AP of DyGFormer, FreeDyG, and ContextDyG, together with the number of test samples in each group. ContextDyG achieves the best performance across all groups. The gains are particularly clear in the medium- and high-context groups, where predictions depend more on neighborhood-level evidence. This indicates that ContextDyG can effectively model shared-neighbor and bridge-neighbor interaction patterns. Meanwhile, ContextDyG also performs consistently well across different pair-frequency groups, showing that it benefits not only from repeated interactions but also from contextual information around the node pair.

These results suggest that contextual interaction modeling provides useful signals for dynamic link prediction, especially when the formation of links is influenced by both repeated historical interactions and broader neighborhood structures.

5 Conclusion

In this paper, we propose ContextDyG, a continuous-time dynamic graph learning method for link prediction. ContextDyG introduces role-aware contextual interaction frequency encoding to characterize the historical roles of neighbors from local, cross-neighborhood, temporal, and frequency-aware perspectives. It further adopts pair-conditioned feature fusion to adaptively integrate node, edge, temporal, and contextual interaction representations.

Extensive experiments on five real-world dynamic graph datasets show that ContextDyG achieves competitive performance under transductive and inductive settings with random, historical, and inductive negative sampling strategies. The ablation studies verify the contribution of each component, while the efficiency analysis shows that ContextDyG improves predictive performance with

moderate computational cost. In addition, the group-wise case study demonstrates that ContextDyG is particularly effective when link prediction depends on contextual neighborhood patterns.

In future work, we plan to extend ContextDyG to more dynamic graph tasks, such as dynamic node classification and temporal event forecasting, and further investigate scalable contextual encoding for larger streaming graphs.

Acknowledgments. We appreciate all the anonymous reviewers for their invaluable comments and suggestions. This work is supported by Talent Program Project of the Institute of Information Engineering, Chinese Academy of Sciences (No.E5YY07111K4). Any opinions, findings and conclusions in this paper are those of the authors and do not necessarily reflect the views of the funding agencies.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Chang, X., Liu, X., Wen, J., Li, S., Fang, Y., Song, L., Qi, Y.: Continuous-time dynamic graph learning via neural interaction processes. In: Proceedings of the 29th ACM international conference on information & knowledge management. pp. 145–154 (2020)
2. Huang, Z., Sun, Y., Wang, W.: Learning continuous system dynamics from irregularly-sampled partial observations. *Advances in Neural Information Processing Systems* **33**, 16177–16187 (2020)
3. Huo, Z., Huang, X., Hu, X.: Link prediction with personalized social influence. In: Proceedings of the AAAI conference on artificial intelligence. vol. 32 (2018)
4. Jin, M., Li, Y.F., Pan, S.: Neural temporal walks: Motif-aware representation learning on continuous-time dynamic graphs. *Advances in Neural Information Processing Systems* **35**, 19874–19886 (2022)
5. Kumar, S., Zhang, X., Leskovec, J.: Predicting dynamic embedding trajectory in temporal interaction networks. In: Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining. pp. 1269–1278 (2019)
6. Li, S., Jin, X., Xuan, Y., Zhou, X., Chen, W., Wang, Y.X., Yan, X.: Enhancing the locality and breaking the memory bottleneck of transformer on time series forecasting. *Advances in neural information processing systems* **32** (2019)
7. Liang, B., Wang, L., Wang, X.: Autoregressive gnn-ode gru model for network dynamics. *arXiv preprint arXiv:2211.10594* (2022)
8. Luo, L., Haffari, G., Pan, S.: Graph sequential neural ode process for link prediction on dynamic and sparse graphs. In: Proceedings of the sixteenth ACM international conference on web search and data mining. pp. 778–786 (2023)
9. Pareja, A., Domeniconi, G., Chen, J., Ma, T., Suzumura, T., Kanezashi, H., Kaler, T., Schardl, T., Leiserson, C.: Evolvegcnn: Evolving graph convolutional networks for dynamic graphs. In: Proceedings of the AAAI conference on artificial intelligence. vol. 34, pp. 5363–5370 (2020)
10. Poursafaei, F., Huang, S., Pelrine, K., Rabbany, R.: Towards better evaluation for dynamic link prediction. *Advances in Neural Information Processing Systems* **35**, 32928–32941 (2022)

11. Rossi, E., Chamberlain, B., Frasca, F., Eynard, D., Monti, F., Bronstein, M.: Temporal graph networks for deep learning on dynamic graphs. *arXiv preprint arXiv:2006.10637* (2020)
12. Song, W., Xiao, Z., Wang, Y., Charlin, L., Zhang, M., Tang, J.: Session-based social recommendation via dynamic graph attention networks. In: *Proceedings of the Twelfth ACM international conference on web search and data mining*. pp. 555–563 (2019)
13. Tian, Y., Qi, Y., Guo, F.: Freedyg: Frequency enhanced continuous-time dynamic graph model for link prediction. In: *The Twelfth International Conference on Learning Representations* (2024)
14. Trivedi, R., Farajtabar, M., Biswal, P., Zha, H.: Dyrep: Learning representations over dynamic graphs. In: *International conference on learning representations* (2019)
15. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. *Advances in neural information processing systems* **30** (2017)
16. Wang, L., Chang, X., Li, S., Chu, Y., Li, H., Zhang, W., He, X., Song, L., Zhou, J., Yang, H.: Tcl: Transformer-based dynamic graph modelling via contrastive learning. *arXiv preprint arXiv:2105.07944* (2021)
17. Wang, X., Lyu, D., Li, M., Xia, Y., Yang, Q., Wang, X., Wang, X., Cui, P., Yang, Y., Sun, B., et al.: Apan: Asynchronous propagation attention network for real-time temporal graph embedding. In: *Proceedings of the 2021 international conference on management of data*. pp. 2628–2638 (2021)
18. Wang, Y., Chang, Y.Y., Liu, Y., Leskovec, J., Li, P.: Inductive representation learning in temporal networks via causal anonymous walks. *arXiv preprint arXiv:2101.05974* (2021)
19. Wu, H., Hu, T., Liu, Y., Zhou, H., Wang, J., Long, M.: Timesnet: Temporal 2d-variation modeling for general time series analysis. *arXiv preprint arXiv:2210.02186* (2022)
20. Wu, H., Xu, J., Wang, J., Long, M.: Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. *Advances in neural information processing systems* **34**, 22419–22430 (2021)
21. Wu, Z., Liu, X., Zhang, X.: Multi dynamic temporal representation graph convolutional network for traffic flow prediction. *Scientific Reports* **15**(1), 1–13 (2025)
22. Xu, D., Ruan, C., Korpeoglu, E., Kumar, S., Achan, K.: Inductive representation learning on temporal graphs. *arXiv preprint arXiv:2002.07962* (2020)
23. Yang, M., Zhou, M., Kalander, M., Huang, Z., King, I.: Discrete-time temporal network embedding via implicit hierarchical learning in hyperbolic space. In: *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining*. pp. 1975–1985 (2021)
24. Yu, L., Sun, L., Du, B., Lv, W.: Towards better dynamic graph learning: New architecture and unified library. *Advances in Neural Information Processing Systems* **36**, 67686–67700 (2023)
25. Zhao, L., Song, Y., Zhang, C., Liu, Y., Wang, P., Lin, T., Deng, M., Li, H.: T-gcn: A temporal graph convolutional network for traffic prediction. *IEEE transactions on intelligent transportation systems* **21**(9), 3848–3858 (2019)
26. Zhou, H., Zhang, S., Peng, J., Zhang, S., Li, J., Xiong, H., Zhang, W.: Informer: Beyond efficient transformer for long sequence time-series forecasting. In: *Proceedings of the AAAI conference on artificial intelligence*. vol. 35, pp. 11106–11115 (2021)